

GRASP E GoF

- GRASP sono principi generali che guidano le scelte progettuali attraverso l'assegnamento di responsabilità.
- GoF sono soluzioni tipiche a problemi ricorrenti, sono idee di progettazione avanzate (Vedremo i Pattern).

Nella progettazione orientata agli oggetti, per definire una prima versione di class diagram, bisogna anzitutto individuare le classi di analisi.

Le classe di analisi hanno tutte/sempre la stessa funzione?

Ovviamente NO!

BCE

Il modello BCE serve a **classificare le classi** del sistema in tre categorie principali, ognuna con una responsabilità ben distinta: **Boundary Control Entity**.

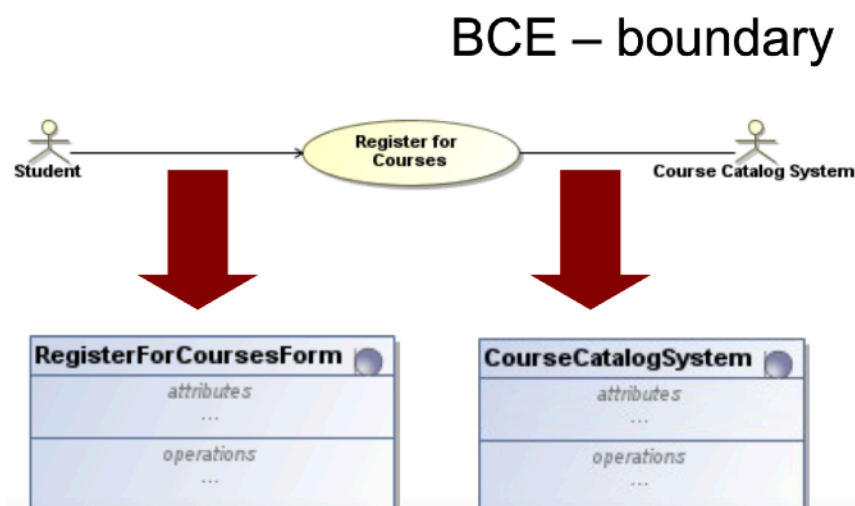
- **Boundary**: mediano l'interazione tra il sistema e l'ambiente, rappresentano elementi al confine del sistema e regolano l'interazione con gli attori. **Si hanno tante classi boundary quante sono le coppie (attore, caso d'uso) all'interno dello Use-Case diagram.**
In sostanza, è un cuscinetto che viene utilizzato dall'utente per comunicare le sue interazioni con il

sistema. Sono la realizzazione delle funzionalità così come viste dal **bordo del sistema**.

- **Control**: coordinano il comportamento durante l'esecuzione di un caso d'uso del sistema e rappresentano comportamenti dipendenti dall'interazione con il sistema. Si hanno tante classi control quanti sono i casi d'uso all'interno dello *use case diagram*.
- **Entity**: rappresentano le **astrazioni chiave del sistema**, modellano il comportamento di un'entità di dominio incapsulando un insieme coeso di dati e sono indipendenti dall'ambiente di esecuzioni. In generale, non comprendono gli attori.

In definitiva, il modello BCE dà luogo a un **diagramma delle classi di analisi** che, solo con raffinamenti successivi, porta alla produzione di un diagramma delle classi di progettazione.

ESEMPIO:



Lo studente attiva il caso d'uso.

Le **classi boundary** individuate sono:

- RegisterForCoursesForm → la maschera/modulo con cui lo studente interagisce.
- CourseCatalogSystem → interfaccia verso il sistema esterno di catalogo corsi.

Lo **studente** non comunica direttamente con le entità, ma attraverso la **boundary RegisterForCoursesForm**.

Il **sistema interno** non si collega direttamente al catalogo, ma usa la **boundary CourseCatalogSystem**.

Se la responsabilità primaria di una boundary è quella di mediare l'interazione tra un attore e il sistema, e se l'associazione in un caso d'uso rappresenta la partecipazione di un attore a quel caso d'uso, la classe boundary rappresenta questa interazione. Per ogni coppia (attore, caso d'uso) nel mio UseCaseDiagram, dovrò andare a creare delle boundary.

Entrambe hanno spazi per **attributi** e **operazioni**, che andranno definiti in base ai requisiti (es. campi del form, metodi per inviare richieste, ecc.).

L'insieme di operazioni che inseriamo nella boundary sono tutte quelle operazioni che l'attore può attivare per richiedere la funzionalità prevista dal caso d'uso.

Può anche essere vista diversamente, come dei messaggi che il sistema ritorna all'utente. L'utente vuole la lista dei corsi e il catalogo ritorna i corsi.

L'obiettivo della boundary è collezionare l'insieme di operazioni per fare in modo che l'attore invochi il caso d'uso, e fornire delle operazioni, in modo tale che, se ci sono delle notifiche dal sistema verso l'attore, il sistema riesce a comunicare con l'attore.

L'obiettivo è specificare i set di operazioni per quell'attore per quel caso d'uso.

È proprio per questo il motivo per il quale per ogni coppia (attore, caso d'uso) abbiamo una boundary.

La boundary definirà la lista delle funzionalità a cui quell'attore può accedere per partecipare a quel caso d'uso: un insieme di operazioni che l'attore può invocare per interagire con il caso d'uso di specie.

Quali sono le cose che l'attore può fare? Quali sono le funzionalità che lo studente può

attivare mentre partecipa al caso d'uso?

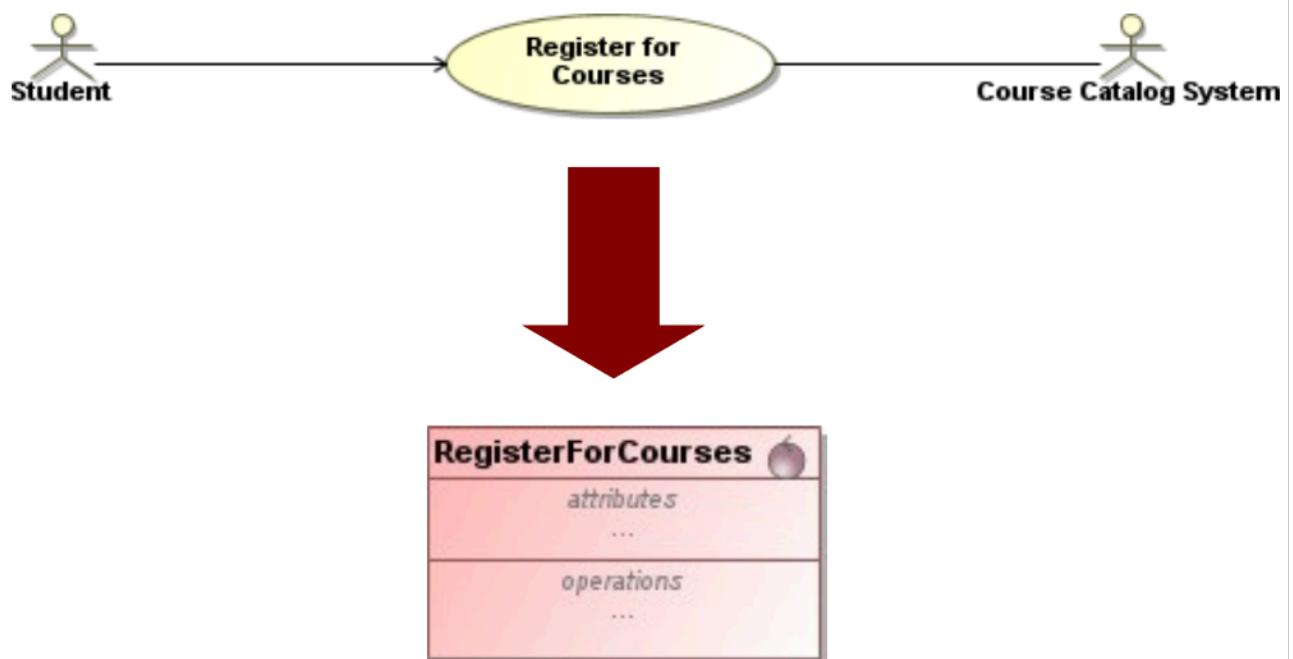
Lo studente vorrebbe registrarsi al corso di ISPW e interagisce con il caso d'uso, non sta chiedendo di fare l'iscrizione al secondo anno.

E l'iscrizione non fa parte delle operazioni della boundary di questo caso d'uso.

Quindi ricapitolando, le operazioni in una boundary sono tutte quelle operazioni che servono per far partecipare l'attore a quel caso d'uso.

Consiglio: enfatizzare le informazioni e le operazioni necessarie per l'interazione con l'attore, non concentrarsi su dettagli implementativi della GUI.

BCE – control



Il controllore GRASP diceva di mettere un'etichetta control ad una classe che ha la **responsabilità** di **coordinare** un'insieme di sottoclassi. Il controller in BCE raffina questo concetto.

Le classi con etichetta controller hanno la responsabilità di coordinare il comportamento durante l'esecuzione di un caso d'uso del sistema.

In sostanza, hanno la responsabilità di realizzare il flusso principale del caso d'uso.

Il numero di classi control è pari al numero dei casi d'uso all'interno dello use case diagram. Il controller ha la logica applicativa che serve per soddisfare il caso d'uso.

Attualmente il sistema è in grado di ricevere l'input dall'esterno (grazie alle boundary) o in grado di produrre uno stimolo verso l'esterno (sempre grazie alle boundary).

Adesso dobbiamo implementare realmente gli internal steps del caso d'uso considerato.

Quindi, per ogni caso d'uso, la classe control ha la responsabilità dell'implementazione del flusso principale che è stato definito nello use case diagram.

Il controller non deve interagire con l'attore esterno. Tipica domanda d'esame.

Quindi, ripeto, per ogni caso d'uso esiste una classe controller che ha la responsabilità di realizzarlo.

Quindi, le boundary raccolgono le informazioni dagli attori, le passano al controller, e questo ultimo elabora queste informazioni per produrre un risultato agli utenti stessi. Il risultato, il controller lo fa ritornare attraverso le boundary.

Le classi boundary e le classi controller possono interagire più volte tra di loro per adempiere una funzionalità all'utente.

Nello svolgimento di un caso d'uso ci possono essere N interazioni tra gli attori e il sistema.

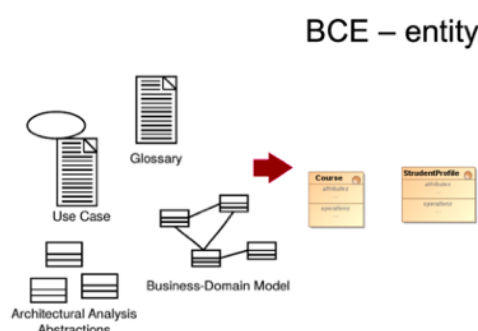
Caso d'uso: Preleva al bancomat.

Metto la tessera, metto pin, scelgo ammontare, conferma, ricevuta o no, ritiro soldi.

Ci sono tante interazioni per un singolo caso d'uso.

Qualsiasi interazione con l'esterno è mediata dalle classi boundary: se il controller ha bisogno di un pin dovrà invocare l'operazione della boundary "richiediPinUtente()".

Adesso, allo stato attuale questa boundary e questo controller si scambiano informazioni: le informazioni che si scambiano sono quelle specifiche per il caso d'uso.



Le entity invece sono le astrazioni chiavi del sistema utili per caratterizzare la trasposizione logica del dominio del sistema che stiamo rappresentando.

Quindi, ricapitolando, il pattern BCE è un pattern di analisi che ci consente di trasformare viste a use case in una prima bozza di vista logica, ossia quali sono i tipi di dato che ci aiutano a rappresentare il nostro sistema.

L'idea è quella di introdurre delle classi che ci permettano di effettuare il passaggio dalla vista a use case in una prima struttura logica del sistema, dove ogni classe ha una responsabilità.

Notazione de Angelis: *La freccia dall'attore al caso d'uso unidirezionale non sta a significare che l'attore non può ricevere dati, ma che l'attore avvia il caso d'uso.* Le comunicazioni sono comunque bidirezionali.

Domanda d'esame.

Dato un sistema con una vista Use Case, applico BCE, quante classi controller mi devo aspettare?

1 classe controller per ogni use case previsto nel sistema.

Nel controller specifico bene gli **internal steps** che servono per soddisfare quel caso d'uso.

Poi è un'operazione della boundary che invocherà le operazioni del controller. **L'insieme delle operazioni del controller sono gli internal steps del flusso principale.**

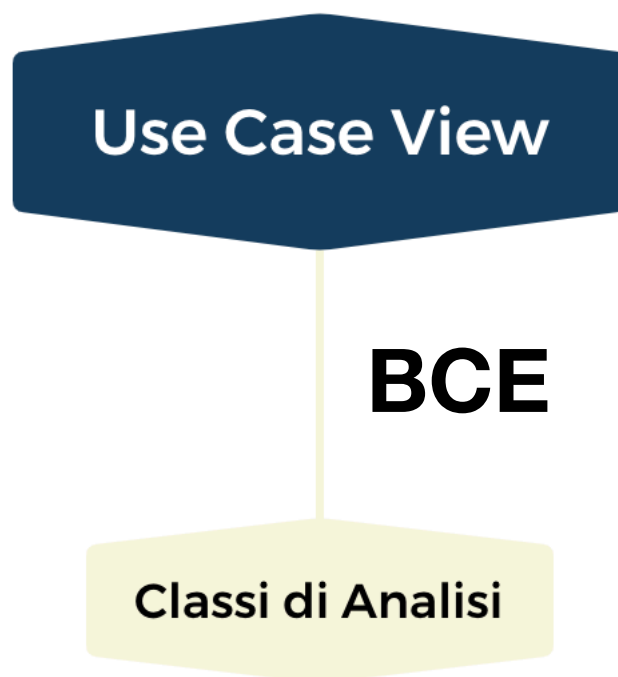
Le operazioni del controller corrispondono proprio agli **internal steps** del flusso principale del caso d'uso: sono le azioni che coordinano boundary ed entity per realizzare lo scenario.

Rigida assunzione che poi non avrà più valenza.

Ora, controller e boundary si scambiano le entity.

Queste entità parlano col controller.

Questo è un passo importante, questa cosa non sarà più valida tra poco. Ora siamo nel passo tra Use Case a class diagram, quindi l'assunzione è lecita.



Questo è il grafico riassuntivo rispetto a quanto detto.

Ora cominciamo ad uscire dal mondo dell'analisi e cominciamo a calare nel mondo della **progettazione**.

Useremo il termine VOPC per indicare un diagramma delle classi risultante a seguito dell'applicazione BCE.

DIAGRAMMA VOPC

VOPC è l'abbreviazione di *View Of Participating Classes*, e descrive le classi che costituiscono un **singolo caso d'uso**.

Generalmente viene utilizzato dopo aver descritto il flusso degli eventi che caratterizzano il relativo caso d'uso.